Defining Query Methods

Defining Query Methods

The repository proxy has two ways to derive a store-specific query from the method name:

- By deriving the query from the method name directly.
- By using a manually defined query.

Available options depend on the actual store. However, there must be a strategy that decides what actual query is created. The next section describes the available options.

Query Lookup Strategies

The following strategies are available for the repository infrastructure to resolve the query. With XML configuration, you can configure the strategy at the namespace through the `query-lookup-strategy` attribute. For Java configuration, you can use the `queryLookupStrategy` attribute of the `EnableJpaRepositories` annotation. Some strategies may not be supported for particular datastores.

- `CREATE` attempts to construct a store-specific query from the query method name. The general approach is to remove a given set of well known prefixes from the method name and parse the rest of

the method. You can read more about query construction in “Query Creation”.

- `USE_DECLARED_QUERY` tries to find a declared query and throws an exception if it cannot find one. The query can be defined by an annotation somewhere or declared by other means. See the documentation of the specific store to find available options for that store. If the repository infrastructure does not find a declared query for the method at bootstrap time, it fails.
- `CREATE_IF_NOT_FOUND` (the default) combines `CREATE` and `USE_DECLARED_QUERY`. It looks up a declared query first, and, if no declared query is found, it creates a custom method name-based query. This is the default lookup strategy and, thus, is used if you do not configure anything explicitly. It allows quick query definition by method names but also custom-tuning of these queries by introducing declared queries as needed.

Query Creation

The query builder mechanism built into the Spring Data repository infrastructure is useful for building constraining queries over entities of the repository.

The following example shows how to create a number of queries:

Query creation from method names

```java
interface PersonRepository extends Repository<Person, Long> {

  List<Person> findByEmailAddressAndLastname(EmailAddress emailAddress, String lastname);

  // Enables the distinct flag for the query
  List<Person> findDistinctPeopleByLastnameOrFirstname(String lastname, String firstname);
  List<Person> findPeopleDistinctByLastnameOrFirstname(String lastname, String firstname);

  // Enabling ignoring case for an individual property
  List<Person> findByLastnameIgnoreCase(String lastname);
  // Enabling ignoring case for all suitable properties
  List<Person> findByLastnameAndFirstnameAllIgnoreCase(String lastname, String firstname);

  // Enabling static ORDER BY for a query
  List<Person> findByLastnameOrderByFirstnameAsc(String lastname);
  List<Person> findByLastnameOrderByFirstnameDesc(String lastname);
}
```

Parsing query method names is divided into subject and predicate. The first part (`find…By`, `exists…By`) defines the subject of the query, the second part forms the predicate. The introducing clause (subject) can contain further expressions. Any text between `find` (or other introducing keywords) and `By` is con-

sidered to be descriptive unless using one of the result-limiting keywords such as a `Distinct` to set a distinct flag on the query to be created or `Top` / `First` to limit query results.

The appendix contains the full list of query method subject keywords and query method predicate keywords including sorting and letter-casing modifiers. However, the first `By` acts as a delimiter to indicate the start of the actual criteria predicate. At a very basic level, you can define conditions on entity properties and concatenate them with `And` and `Or`.

The actual result of parsing the method depends on the persistence store for which you create the query. However, there are some general things to notice:

- The expressions are usually property traversals combined with operators that can be concatenated. You can combine property expressions with `AND` and `OR`. You also get support for operators such as `Between`, `LessThan`, `GreaterThan`, and `Like` for the property expressions. The supported operators can vary by datastore, so consult the appropriate part of your reference documentation.
- The method parser supports setting an `IgnoreCase` flag for individual properties (for example, `findByLastnameIgnoreCase(…)`) or for all properties of a type that supports ignoring case (usually `String` instances — for example, `findByLastnameAndFirstnameAllIgnoreCase(…)`). Whether ignoring cases is supported may vary by store, so consult the relevant sections in the reference documentation for the store-specific query method.
- You can apply static ordering by appending an `OrderBy` clause to the query method that references a property and by providing a sorting direction (`Asc` or `Desc`). To create a query method that supports dynamic sorting, see "Paging, Iterating Large Results, Sorting & Limiting".

Reserved Method Names

While derived repository methods bind to properties by name, there are a few exceptions to this rule when it comes to certain method names inherited from the base repository targeting the *identifier* property. Those *reserved methods* like `CrudRepository#findById` (or just `findById`) are targeting the *identifier* property regardless of the actual property name used in the declared method.

Consider the following domain type holding a property `pk` marked as the identifier via `@Id` and a property called `id`. In this case you need to pay close attention to the naming of your lookup methods as they may collide with predefined signatures:

```java
class User {
  @Id Long pk;

  Long id;
```

```
  // …
}

interface UserRepository extends Repository<User, Long> {

  Optional<User> findById(Long id);      3

  Optional<User> findByPk(Long pk);      4

  Optional<User> findUserById(Long id);  5
}
```

1 The identifier property (primary key).

2 A property named `id`, but not the identifier.

3 Targets the `pk` property (the one marked with `@Id` which is considered to be the identifier) as it refers to a `CrudRepository` base repository method. Therefore, it is not a derived query using of `id` as the property name would suggest because it is one of the *reserved methods*.

4 Targets the `pk` property by name as it is a derived query.

5 Targets the `id` property by using the descriptive token between `find` and `by` to avoid collisions with *reserved methods*.

This special behaviour not only targets lookup methods but also applies to the `exits` and `delete` ones. Please refer to the "Repository query keywords" for the list of methods.

Property Expressions

Property expressions can refer only to a direct property of the managed entity, as shown in the preceding example. At query creation time, you already make sure that the parsed property is a property of the managed domain class. However, you can also define constraints by traversing nested properties. Consider the following method signature:

```
List<Person> findByAddressZipCode(ZipCode zipCode);
```
JAVA

Assume a `Person` has an `Address` with a `ZipCode`. In that case, the method creates the `x.address.zipCode` property traversal. The resolution algorithm starts by interpreting the entire part (`AddressZipCode`) as the property and checks the domain class for a property with that name (uncapitalized). If the algorithm succeeds, it uses that property. If not, the algorithm splits up the source at the camel-case parts from the right side into a head and a tail and tries to find the corresponding property — in our example, `AddressZip` and `Code`. If the algorithm finds a property with that head, it takes the

tail and continues building the tree down from there, splitting the tail up in the way just described. If the first split does not match, the algorithm moves the split point to the left (`Address` , `ZipCode`) and continues.

Although this should work for most cases, it is possible for the algorithm to select the wrong property. Suppose the `Person` class has an `addressZip` property as well. The algorithm would match in the first split round already, choose the wrong property, and fail (as the type of `addressZip` probably has no `code` property).

To resolve this ambiguity you can use `_` inside your method name to manually define traversal points. So our method name would be as follows:

```
List<Person> findByAddress_ZipCode(ZipCode zipCode);
```
JAVA

NOTE

Because we treat underscores (`_`) as a reserved character, we strongly advise to follow standard Java naming conventions (that is, not using underscores in property names but applying camel case instead).

CAUTION

Field Names starting with underscore:
Field names may start with underscores like `String _name` . Make sure to preserve the `_` as in `_name` and use double `_` to split nested paths like `user__name` .

Upper Case Field Names:
Field names that are all uppercase can be used as such. Nested paths if applicable require splitting via `_` as in `USER_name` .

Field Names with 2nd uppercase letter:
Field names that consist of a starting lower case letter followed by an uppercase one like `String qCode` can be resolved by starting with two upper case letters as in `QCode` . Please be aware of potential path ambiguities.

Path Ambiguities:
In the following sample the arrangement of properties `qCode` and `q` , with `q` containing a property called `code` , creates an ambiguity for the path `QCode` .

```
record Container(String qCode, Code q) {}
record Code(String code) {}
```

Since a direct match on a property is considered first, any potential nested paths will not be considered and the algorithm picks the `qCode` field. In order to select the `code` field in `q` the underscore notation `Q_Code` is required.

Repository Methods Returning Collections or Iterables

Query methods that return multiple results can use standard Java `Iterable`, `List`, and `Set`. Beyond that, we support returning Spring Data's `Streamable`, a custom extension of `Iterable`, as well as collection types provided by Vavr. Refer to the appendix explaining all possible query method return types.

Using Streamable as Query Method Return Type

You can use `Streamable` as alternative to `Iterable` or any collection type. It provides convenience methods to access a non-parallel `Stream` (missing from `Iterable`) and the ability to directly `….filter(…)` and `….map(…)` over the elements and concatenate the `Streamable` to others:

Using Streamable to combine query method results

JAVA

```
interface PersonRepository extends Repository<Person, Long> {
  Streamable<Person> findByFirstnameContaining(String firstname);
  Streamable<Person> findByLastnameContaining(String lastname);
}

Streamable<Person> result = repository.findByFirstnameContaining("av")
  .and(repository.findByLastnameContaining("ea"));
```

Returning Custom Streamable Wrapper Types

Providing dedicated wrapper types for collections is a commonly used pattern to provide an API for a query result that returns multiple elements. Usually, these types are used by invoking a repository method returning a collection-like type and creating an instance of the wrapper type manually. You can avoid that additional step as Spring Data lets you use these wrapper types as query method return types if they meet the following criteria:

1. The type implements `Streamable`.
2. The type exposes either a constructor or a static factory method named `of(…)` or `valueOf(…)` that takes `Streamable` as an argument.

The following listing shows an example:

JAVA

```
class Product {                                          1
  MonetaryAmount getPrice() { … }
}

@RequiredArgsConstructor(staticName = "of")
class Products implements Streamable<Product> {          2
```

```
  private final Streamable<Product> streamable;

  public MonetaryAmount getTotal() {                    3
    return streamable.stream()
      .map(Product::getPrice)
      .reduce(Money.of(0), MonetaryAmount::add);
  }


  @Override
  public Iterator<Product> iterator() {                 4
    return streamable.iterator();
  }
}

interface ProductRepository implements Repository<Product, Long> {
  Products findAllByDescriptionContaining(String text); 5
}
```

1 A `Product` entity that exposes API to access the product's price.

2 A wrapper type for a `Streamable<Product>` that can be constructed by using `Products.of(…)` (factory method created with the Lombok annotation). A standard constructor taking the `Streamable<Product>` will do as well.

3 The wrapper type exposes an additional API, calculating new values on the `Streamable<Product>`.

4 Implement the `Streamable` interface and delegate to the actual result.

5 That wrapper type `Products` can be used directly as a query method return type. You do not need to return `Streamable<Product>` and manually wrap it after the query in the repository client.

Support for Vavr Collections

Vavr is a library that embraces functional programming concepts in Java. It ships with a custom set of collection types that you can use as query method return types, as the following table shows:

Vavr collection type	Used Vavr implementation type	Valid Java source types
`io.vavr.collection.Seq`	`io.vavr.collection.List`	`java.util.Iterable`
`io.vavr.collection.Set`	`io.vavr.collection.LinkedHashSet`	`java.util.Iterable`
`io.vavr.collection.Map`	`io.vavr.collection.LinkedHashMap`	`java.util.Map`

You can use the types in the first column (or subtypes thereof) as query method return types and get the types in the second column used as implementation type, depending on the Java type of the actual query result (third column). Alternatively, you can declare `Traversable` (the Vavr `Iterable` equivalent), and we then derive the implementation class from the actual return value. That is, a `java.util.List` is turned into a Vavr `List` or `Seq`, a `java.util.Set` becomes a Vavr `LinkedHashSet` `Set`, and so on.

Streaming Query Results

You can process the results of query methods incrementally by using a Java 8 `Stream<T>` as the return type. Instead of wrapping the query results in a `Stream`, data store-specific methods are used to perform the streaming, as shown in the following example:

Stream the result of a query with Java 8 `Stream<T>`

```java
@Query("select u from User u")
Stream<User> findAllByCustomQueryAndStream();

Stream<User> readAllByFirstnameNotNull();

@Query("select u from User u")
Stream<User> streamAllPaged(Pageable pageable);
```

NOTE

A `Stream` potentially wraps underlying data store-specific resources and must, therefore, be closed after usage. You can either manually close the `Stream` by using the `close()` method or by using a Java 7 `try-with-resources` block, as shown in the following example:

Working with a `Stream<T>` result in a `try-with-resources` block

```java
try (Stream<User> stream = repository.findAllByCustomQueryAndStream()) {
  stream.forEach(…);
}
```

NOTE

Not all Spring Data modules currently support `Stream<T>` as a return type.

Asynchronous Query Results

You can run repository queries asynchronously by using Spring's asynchronous method running capability. This means the method returns immediately upon invocation while the actual query occurs in a task

that has been submitted to a Spring `TaskExecutor`. Asynchronous queries differ from reactive queries and should not be mixed. See the store-specific documentation for more details on reactive support. The following example shows a number of asynchronous queries:

```java
@Async
Future<User> findByFirstname(String firstname);               1

@Async
CompletableFuture<User> findOneByFirstname(String firstname); 2
```

1 Use `java.util.concurrent.Future` as the return type.

2 Use a Java 8 `java.util.concurrent.CompletableFuture` as the return type.

Paging, Iterating Large Results, Sorting & Limiting

To handle parameters in your query, define method parameters as already seen in the preceding examples. Besides that, the infrastructure recognizes certain specific types like `Pageable`, `Sort` and `Limit`, to apply pagination, sorting and limiting to your queries dynamically. The following example demonstrates these features:

Using `Pageable`, `Slice`, `ScrollPosition`, `Sort` and `Limit` in query methods

```java
Page<User> findByLastname(String lastname, Pageable pageable);

Slice<User> findByLastname(String lastname, Pageable pageable);

Window<User> findTop10ByLastname(String lastname, ScrollPosition position, Sort sort);

List<User> findByLastname(String lastname, Sort sort);

List<User> findByLastname(String lastname, Sort sort, Limit limit);

List<User> findByLastname(String lastname, Pageable pageable);
```

IMPORTANT

APIs taking `Sort`, `Pageable` and `Limit` expect non-`null` values to be handed into methods. If you do not want to apply any sorting or pagination, use `Sort.unsorted()`, `Pageable.unpaged()` and `Limit.unlimited()`.

The first method lets you pass an `org.springframework.data.domain.Pageable` instance to the query method to dynamically add paging to your statically defined query. A `Page` knows about the total number of elements and pages available. It does so by the infrastructure triggering a count query to calcu-

late the overall number. As this might be expensive (depending on the store used), you can instead return a `Slice`. A `Slice` knows only about whether a next `Slice` is available, which might be sufficient when walking through a larger result set.

Sorting options are handled through the `Pageable` instance, too. If you need only sorting, add an `org.springframework.data.domain.Sort` parameter to your method. As you can see, returning a `List` is also possible. In this case, the additional metadata required to build the actual `Page` instance is not created (which, in turn, means that the additional count query that would have been necessary is not issued). Rather, it restricts the query to look up only the given range of entities.

NOTE

To find out how many pages you get for an entire query, you have to trigger an additional count query. By default, this query is derived from the query you actually trigger.

IMPORTANT

Special parameters may only be used once within a query method.
Some special parameters described above are mutually exclusive. Please consider the following list of invalid parameter combinations.

Parameters	Example	Reason
`Pageable` and `Sort`	`findBy…(Pageable page, Sort sort)`	`Pageable` already defines `Sort`
`Pageable` and `Limit`	`findBy…(Pageable page, Limit limit)`	`Pageable` already defines a limit.

The `Top` keyword used to limit results can be used to along with `Pageable` whereas `Top` defines the total maximum of results, whereas the Pageable parameter may reduce this number.

Which Method is Appropriate?

The value provided by the Spring Data abstractions is perhaps best shown by the possible query method return types outlined in the following table below. The table shows which types you can return from a query method

Table 1. Consuming Large Query Results

Method	Amount of Data Fetched	Query Structure	Constraints

Method	Amount of Data Fetched	Query Structure	Constraints
`List<T>`	All results.	Single query.	Query results haust all mem Fetching all d time-intensiv
`Streamable<T>`	All results.	Single query.	Query results haust all mem Fetching all d time-intensiv
`Stream<T>`	Chunked (one-by-one or in batches) depending on `Stream` consumption.	Single query using typically cursors.	Streams must after usage to source leaks.
`Flux<T>`	Chunked (one-by-one or in batches) depending on `Flux` consumption.	Single query using typically cursors.	Store module vide reactive infrastructure
`Slice<T>`	`Pageable.getPageSize() + 1` at `Pageable.getOffset()`	One to many queries fetching data starting at `Pageable.getOffset()` applying limiting.	A `Slice` can gate to the n • `Slice` details v there is to fetch • Offset-b queries inefficie the offs large be databas to mate full resu

Method	Amount of Data Fetched	Query Structure	Constraints
Offset-based `Window<T>`	`limit + 1` at `OffsetScrollPosition.getOffset()`	One to many queries fetching data starting at `OffsetScrollPosition.getOffset()` applying limiting.	A `Window` ca igate to the n `Window`. • `Window` details v there is to fetch • Offset-b queries inefficie the offs large be databas to mate full resu
`Page<T>`	`Pageable.getPageSize()` at `Pageable.getOffset()`	One to many queries starting at `Pageable.getOffset()` applying limiting. Additionally, `COUNT(…)` query to determine the total number of elements can be required.	Often times, queries are re that are costl • Offset-b queries inefficie the offs large be databas to mate full resu

Method	Amount of Data Fetched	Query Structure	Constraints
Keyset-based `Window<T>`	`limit + 1` using a rewritten `WHERE` condition	One to many queries fetching data starting at `KeysetScrollPosition.getKeys()` applying limiting.	A `Window` ca igate to the n `Window`. • `Window` details v there is to fetch • Keyset-l queries proper i structur cient qu • Most da do not v when Ke based q sults co values. • Results pose all keys in t sults rec projecti lect pot more pr than rec the actu

Paging and Sorting

You can define simple sorting expressions by using property names. You can concatenate expressions to collect multiple criteria into one expression.

Defining sort expressions

```java
Sort sort = Sort.by("firstname").ascending()
  .and(Sort.by("lastname").descending());
```

For a more type-safe way to define sort expressions, start with the type for which to define the sort expression and use method references to define the properties on which to sort.

Defining sort expressions by using the type-safe API

```java
TypedSort<Person> person = Sort.sort(Person.class);

Sort sort = person.by(Person::getFirstname).ascending()
  .and(person.by(Person::getLastname).descending());
```

NOTE

`TypedSort.by(…)` makes use of runtime proxies by (typically) using CGlib, which may interfere with native image compilation when using tools such as Graal VM Native.

If your store implementation supports Querydsl, you can also use the generated metamodel types to define sort expressions:

Defining sort expressions by using the Querydsl API

```java
QSort sort = QSort.by(QPerson.firstname.asc())
  .and(QSort.by(QPerson.lastname.desc()));
```

Limiting Query Results

In addition to paging it is possible to limit the result size using a dedicated `Limit` parameter. You can also limit the results of query methods by using the `First` or `Top` keywords, which you can use interchangeably but may not be mixed with a `Limit` parameter. You can append an optional numeric value to `Top` or `First` to specify the maximum result size to be returned. If the number is left out, a result size of 1 is assumed. The following example shows how to limit the query size:

Limiting the result size of a query with `Top` and `First`

```java
List<User> findByLastname(String lastname, Limit limit);

User findFirstByOrderByLastnameAsc();

User findTopByLastnameOrderByAgeDesc(String lastname);

Page<User> queryFirst10ByLastname(String lastname, Pageable pageable);

Slice<User> findTop3By(Pageable pageable);

List<User> findFirst10ByLastname(String lastname, Sort sort);

List<User> findTop10ByLastname(String lastname, Pageable pageable);
```

The limiting expressions also support the `Distinct` keyword for datastores that support distinct queries. Also, for the queries that limit the result set to one instance, wrapping the result into with the `Optional` keyword is supported.

If pagination or slicing is applied to a limiting query pagination (and the calculation of the number of available pages), it is applied within the limited result.

NOTE

Limiting the results in combination with dynamic sorting by using a `Sort` parameter lets you express query methods for the 'K' smallest as well as for the 'K' biggest elements.